

When Failure Is Evidence

Separating Instruction from Experiment in Generative AI Prompting

Watson Hartsoe

Final Working Paper · 18 August 2026

Abstract

Prompt engineering is usually framed as a problem of writing instructions that reliably elicit desired outputs. That framing collapses two different practices. Some prompts are directive: they seek an artifact. Others are epistemic: they are interventions designed to reveal how an opaque generative system behaves. Drawing on Kirsh and Maglio's distinction between pragmatic and epistemic action, empirical studies of prompt practice, research on folk theories of changing algorithmic systems, and current platform mechanisms, this paper argues that a substantial part of skilled prompting is better understood as experimental practice under a moving target. Users perturb inputs, inspect consequences, form provisional causal beliefs, exchange heuristics, and revise those beliefs when models change. A generation can therefore fail as a product while succeeding as evidence. This distinction explains why informal prompt rules proliferate, why community knowledge matters, why model updates fossilize old advice, and why production systems increasingly rely on model snapshots, evaluation suites, and automated prompt optimization. The implication is not that language becomes irrelevant. It is that prompt literacy must include experimental literacy: the ability to design discriminating interventions, preserve relevant conditions, compare outcomes, and treat prompt rules as provisional claims rather than timeless incantations.

Keywords: prompt engineering; generative AI; epistemic action; folk theories; experimental practice; algorithmic literacy; evaluation

1. The Wrong Success Criterion

Prompt engineering is commonly evaluated by a simple question: did the instruction produce the desired result? The question is reasonable, but it is incomplete. In everyday use, a person often submits a prompt not because the next output is expected to be final, but because the output will reveal something about what to do next. A writer removes a constraint to see whether a model's tone changes. An image maker swaps one modifier while holding the rest of a prompt steady. A programmer asks for a deliberately narrow transformation to discover whether an agent is respecting repository conventions. In each case, the generated artifact is also an observation about the system that produced it.

The distinction matters because a prompt that fails as an instruction can succeed as an experiment. If removing a phrase causes the composition to collapse, the unwanted image may be more informative than a superficially better one. If a code-generating model violates a constraint only after a tool becomes available, the violation can reveal where the effective boundary of the task lies. A result can be pragmatically poor yet epistemically useful. Treating every undesirable generation as the same kind of failure hides this difference and makes prompt practice look more irrational than it is.

Kirsh and Maglio (1994) offer a vocabulary for this split. In their study of Tetris, they distinguish pragmatic actions, which move an actor toward an external goal, from epistemic actions, which alter the world in order to uncover information or reduce the difficulty of cognition. Players sometimes rotate pieces in ways that do not immediately improve the board. Those rotations make relevant properties easier to perceive or compare. Once informational state is included in the analysis, an apparently wasteful action can become an efficient one. The same distinction can be cautiously extended to generative systems. A directive prompt primarily seeks a useful artifact; an epistemic prompt primarily seeks information that will make later action better.

This paper develops that distinction because it reorganizes several otherwise disconnected observations about contemporary generative AI. It clarifies why users repeatedly modify prompts even when they already know what they want, why prompt communities describe themselves as experimenting, why prompt advice degrades when model versions change, and why current production guidance increasingly couples prompt changes to evaluation suites. The claim is not that all prompting is experimental. It is that instruction and experiment are different functions that are routinely interleaved, and that the experimental function has been underdescribed.

2. Two Functions of a Prompt

Natural-language interfaces invite a conversational interpretation: the user says what they mean, the model responds, and better wording produces better obedience. Studies of non-expert prompt design show why that picture is unstable. Zamfirescu-Pereira et al. (2023), observing people designing LLM-based chatbots, found that participants explored prompt designs opportunistically rather than systematically and often overgeneralized from limited evidence. Expectations imported from human-to-human instruction could become obstacles rather than aids. The important point is not simply that novices wrote weak prompts. It is that they were trying to infer a machine’s response function from sparse, ambiguous feedback while simultaneously using that function to build something.

That is a structurally difficult task. A user changes the words and the output changes, but the observed difference may reflect many causes: the edited phrase, stochastic sampling, hidden system instructions, model updates, retrieval state, tool availability, personalization, or interactions among these factors. The surface action is linguistic, yet the learning problem is causal. The user is asking, implicitly or explicitly, which intervention produced which consequence.

The distinction between directive and epistemic prompting makes this causal work visible. Directive prompting is judged by the usefulness of an output. Epistemic prompting is judged by the information gained from a comparison. The two can occur in the same sequence: a person begins with an artifact goal, encounters an unexpected result, turns the next prompt into a diagnostic intervention, updates a belief about the system, and then returns to artifact production. The loop is not simply “prompt, fail, prompt again.” It alternates between making and learning.

The difference also changes what counts as a good failure. For directive prompting, failure is an output that does not satisfy the task. For epistemic prompting, failure is an intervention whose result cannot discriminate among competing explanations. A bizarre generation can be highly successful if it isolates a variable. A polished generation can be epistemically useless if too many prompt elements changed at once to tell what mattered.

Table 1. An analytic distinction between directive and epistemic prompting

Dimension	Directive prompting	Epistemic prompting
Primary goal	Obtain a useful artifact	Reduce uncertainty about system behavior
Success criterion	Output satisfies task	Result discriminates among explanations
Typical unit	Prompt–output pair	Intervention–comparison
Useful failure	Rare	Outcome that falsifies or narrows a causal belief
Version change	Potential regression	Change in the validity conditions of prior findings
What to preserve	Working prompt and artifact	Intervention, conditions, comparison, result, and interpretation

Note. The distinction is analytic; many real interactions alternate between the two functions.

3. Prompt Communities as Experimental Institutions

The experimental character of prompting is not confined to individual trial and error. Mahdavi Goloujeh, Sullivan, and Magerko (2024) studied nineteen members of the MidJourney community and argued that prompting should be understood as a socially situated practice rather than only an individual technical skill. Their participants described proprietary, derivative, collaborative, and provocative orientations toward prompt sharing and reuse. Most important for the present argument, some participants described organized collective experimentation. One participant explained that a group was trying to push image prompts as far as possible because there was no defining literature telling them where the limits were.

That report changes the status of a prompt community. It is not merely a place where finished recipes circulate. It can also be an informal experimental institution operating against a proprietary and incompletely documented system. Members propose variations, run trials, compare results, share modifiers, and carry discoveries into later work. The collective object is not only a library of good strings. It is a provisional body of causal claims about what the model seems to do under particular conditions.

This helps explain why social circulation is constitutive of prompt expertise. A person can learn from another user’s intervention without copying the final artifact. In Mahdavi Goloujeh et al.’s study, participants describe

modifying others' prompts, adding their own detail, sharing tips, and watching prompts change over time. Prompt knowledge is therefore produced through repeated transformations whose outcomes become evidence for others. The community distributes the cost of probing a large possibility space.

But distributed experimentation also creates a methodological hazard. Informal communities rarely control every variable, and the platforms themselves may not expose the variables that matter. A useful-looking rule can be confounded with seed effects, hidden defaults, personalization, or model version. A memorable anecdote can become a community law. The same social machinery that accelerates discovery can accelerate superstition. The question is therefore not whether folk experimentation occurs, but what distinguishes a portable finding from a locally successful ritual.

4. The Moving Target Problem

DeVito's (2021) account of adaptive folk theorization provides a second piece of the mechanism. Studying users of opaque algorithmic social platforms, DeVito describes folk theories as intuitive, informal accounts of causal relations that help people act despite incomplete technical knowledge. Such theories can remain useful without full mechanistic detail, but they must change when the platform changes. The study distinguishes levels of theorization from basic awareness and causal powers to mechanistic fragments and mechanistic ordering. Structural theorization, in which users recognize multiple causal factors and their possible relations, is associated with greater ability to experiment and adapt.

Generative AI intensifies the same problem because the experimental object is explicitly versioned and frequently replaced. Midjourney's current documentation, for example, records V8.1 as the default from June 10 through July 23, 2026, with V8.2 replacing it on July 24. The company describes the versions as differing in aesthetics, image quality, personalization, speed, and prompt adherence. A prompting rule discovered under one version is therefore not automatically a rule about the next. The text can remain unchanged while the system that gives the text operational meaning changes underneath it.

This produces what might be called prompt fossils: preserved rules whose originating conditions no longer obtain. "Put the style word first." "Repeat the subject." "Use this punctuation." Such advice can circulate long after the version, preprocessing behavior, default style, or interaction pattern that made it useful has changed. The rule survives as language while its causal support has disappeared.

The moving-target problem also sharpens the difference between rhetoric and evidence. If a practitioner treats a prompt tip as an incantation, contradictory results appear as fickleness. If the tip is treated as a hypothesis indexed to a model and context, contradictory results become a reason to retest. The second posture does not require internal access to the model. It requires only enough experimental discipline to know that a rule learned from one environment is provisional in another.

5. From Magic Words to Experimental Literacy

Current production practice is beginning to formalize this experimental posture. OpenAI's prompt-engineering documentation recommends pinning applications to model snapshots when consistency matters and building tests and evaluation suites that can monitor prompt behavior as prompts are iterated or models are changed. Its evaluation guidance treats systematic testing against specified criteria as essential to reliable model applications, especially when upgrading or trying new models (OpenAI, 2026a, 2026b). The point is not vendor-specific. The underlying move is from remembering that a prompt "worked" to specifying how that claim will be tested again.

This move changes the locus of expertise. If success depends on a single lucky wording, the expert is a rhetorician of the model: someone who knows the incantation. If success depends on comparative tests, stable task criteria, model snapshots, representative examples, and the interpretation of regressions, the expert increasingly resembles an experimental designer. The durable skill is not possession of a particular string but the ability to create conditions under which competing prompt variants can be meaningfully compared.

Automated prompt generation makes this shift clearer. OpenAI's current Playground documentation describes meta-prompts that generate or improve prompts from a task description and notes possible integration of more advanced optimization approaches (OpenAI, 2026c). When the system can propose the operative wording, the human contribution does not disappear; it moves upward. Someone must define the task, provide

examples, decide what counts as success, select an evaluation set, and judge whether an optimized prompt has generalized or merely overfit the test cases.

This suggests a narrower and stronger definition of prompt literacy. Prompt literacy is not primarily the ability to recall effective phrases. It is the ability to conduct disciplined interaction with a partially known computational system: to vary inputs deliberately, compare outputs, notice confounds, preserve relevant conditions, and revise beliefs when the evidence changes. Natural language remains the intervention surface, but experimental reasoning determines what can responsibly be learned from it.

6. Beyond Words

The experimental interpretation also survives when language stops being the primary control channel. Midjourney's Style Creator offers a revealing case. Users shape a reusable style reference by choosing among presented images; the system learns from both selected and unselected samples. Midjourney explicitly notes that the text entered in the Style Creator is used to generate preview images but does not itself guide the style-construction process (Midjourney, 2026b). The user's operative contribution is a sequence of discriminations: this, not that; closer, not farther.

This matters because it separates specification from verbalization. A user can reliably recognize a desired aesthetic without being able to describe the rule that unites the preferred examples. Repeated selection turns tacit discrimination into an executable representation. The process is still experimental: each refinement round presents consequences, the user judges them, and the system updates. But the intervention is no longer reducible to a sentence.

The case exposes a weakness in definitions of prompt engineering that are too tightly attached to wording. What persists across text prompting, preference selection, prompt optimization, and evaluation-driven iteration is not a particular linguistic form. It is a feedback structure in which a person or group applies controlled differences, observes system behavior, and uses the resulting evidence to shape subsequent control. Language is one powerful medium for this structure, not its essence.

7. A Research Program for Prompting as Experiment

Treating some prompts as epistemic actions creates empirical questions that current prompt research can test directly. The first is functional: what fraction of real executions are artifact-seeking, information-seeking, or mixed? Prompt logs alone cannot answer this because the same sentence may serve different purposes. A study would need users to state, before or immediately after execution, what would make the generation useful. An output judged "bad" but retained because it changed the next hypothesis would be strong evidence for epistemic prompting.

The second question concerns experimental quality. Do experts change fewer variables at once? Do they reuse comparison cases? Do they separate model variance from prompt variance? Do they preserve outputs that falsify their expectations? Zamfirescu-Pereira et al.'s finding that non-experts explored opportunistically rather than systematically suggests a measurable skill dimension: not simply prompt quality, but intervention quality. Training could then teach comparative design rather than templates alone.

The third question concerns social reliability. Mahdavi Goloujeh et al. show that prompt knowledge is shared, adapted, and collectively explored. A longitudinal study could take a concrete community rule, recover its earliest documented uses, and test it across seeds and model versions. The important outcome would not be whether the rule is "true" in the abstract, but where its effect appears, disappears, reverses, or becomes indistinguishable from noise. Community knowledge would become historically indexed rather than flattened into evergreen advice.

The fourth question concerns interface design. Existing interfaces usually privilege the current prompt and its latest output. An experimental interface would make comparison cheap: branch from an earlier state, vary one element, keep model/version conditions visible, retain rejected outputs, and attach evaluation criteria to a run. These features would not force every creative act into laboratory procedure. They would simply support users when the purpose of an interaction shifts from making an artifact to learning how the system behaves.

8. Limits and Counterarguments

The argument has several limits. Kirsh and Maglio studied embodied action in Tetris, not language-model interaction. Extending the pragmatic/epistemic distinction to prompting is an analytic proposal, not a result established by their experiments. The analogy earns its value only if empirical studies find prompt executions whose primary benefit is information gained rather than artifact produced.

The social evidence is also narrow. Mahdavi Goloujeh et al.'s study concerns nineteen U.S.-based MidJourney users and English-language prompting. Their participants' descriptions show that collaborative experimentation exists, but not how common it is, how reliable its findings are, or whether the same practices characterize coding agents, enterprise LLM systems, or non-English communities. DeVito's research concerns algorithmic social platforms, not generative models; its relevance lies in the shared problem of acting through opaque and changing systems, not in an assumed equivalence.

There is also a genuine countertrend toward stabilization. Model snapshots, evaluation suites, structured prompts, and production controls can make a generative system behave more like an engineered dependency and less like a mysterious interlocutor. In well-bounded applications, prompting may become routine configuration rather than active experimentation. This does not refute the distinction. It identifies the conditions under which the epistemic phase can recede: when uncertainty has been sufficiently reduced, evaluation criteria are stable, and the environment is controlled enough that a prompt can function primarily as an instruction.

Finally, experimental literacy can itself become ritualized. A/B comparisons, eval scores, and version labels do not guarantee causal understanding. An evaluation set can encode the wrong task; repeated testing can overfit; a version number can conceal other changing infrastructure. The experimental framing should therefore increase epistemic humility, not replace folk incantations with statistical ones.

9. Conclusion: The Prompt Is Sometimes a Probe

The central mistake in prompt engineering is not believing that words matter. Words clearly matter. The mistake is assuming that every prompt should be understood by the same success criterion. Some prompts are attempts to make the system produce something. Others are attempts to make the system reveal something. Real practice moves between these functions continuously.

Once the distinction is visible, several puzzling features of generative AI become easier to explain. Iteration is not merely repeated failure; it can be information gathering. Prompt communities are not only marketplaces of tricks; they can be distributed sites of empirical inquiry. Prompt advice decays because its object changes. Model snapshots and evals matter because the validity of a prompt rule depends on conditions. Preference-based systems matter because experimental specification can operate through selection even when words cannot state the desired rule.

The practical consequence is simple. A mature account of prompting should ask not only, "What did these words make?" It should also ask, "What did this intervention let the user learn?" A generation that does not advance the artifact may still advance the informational state of the person or group working with the model. In that case, failure is not the opposite of success. It is evidence.

References

- DeVito, M. A. (2021). Adaptive folk theorization as a path to algorithmic literacy on changing platforms. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW2), Article 339. <https://doi.org/10.1145/3476080>
- Kirsh, D., & Maglio, P. (1994). On distinguishing epistemic from pragmatic action. *Cognitive Science*, 18(4), 513–549. https://doi.org/10.1207/s15516709cog1804_1
- Mahdavi Goloujeh, A., Sullivan, A., & Magerko, B. (2024). The social construction of generative AI prompts. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*. Association for Computing Machinery. <https://doi.org/10.1145/3613905.3650947>
- Midjourney. (2026a). Version. Midjourney Documentation. Accessed August 17, 2026. <https://docs.midjourney.com/hc/en-us/articles/32199405667853-Version>
- Midjourney. (2026b). Style Creator. Midjourney Documentation. Accessed August 17, 2026. <https://docs.midjourney.com/hc/en-us/articles/41308374558221-Style-Creator>
- OpenAI. (2026a). Prompt engineering. OpenAI API Documentation. Accessed August 17, 2026. <https://developers.openai.com/api/docs/guides/prompt-engineering>
- OpenAI. (2026b). Working with evals. OpenAI API Documentation. Accessed August 17, 2026. <https://developers.openai.com/api/docs/guides/evals>
- OpenAI. (2026c). Prompt generation. OpenAI API Documentation. Accessed August 17, 2026. <https://developers.openai.com/api/docs/guides/prompt-generation>
- Zamfirescu-Pereira, J. D., Wong, R. Y., Hartmann, B., & Yang, Q. (2023). Why Johnny can't prompt: How non-AI experts try (and fail) to design LLM prompts. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, Article 437, 1–21. <https://doi.org/10.1145/3544548.3581388>